

Semantic-Topological Cognitive Maps for Assistive Indoor Navigation of a Robot Partner

Anhar Risnumawan^{1,6}, Achmad Fahrul Aji^{2,7}, Raditya Artha Rochmanto^{3,7},
Teuku Zikri Fatahillah⁴, and Naoyuki Kubota⁵

^{1,2,3,4,5}Department of Mechanical Systems Engineering, Graduate School of Systems Design, Tokyo Metropolitan University
6-6 Asahigaoka, Hino, Tokyo 191-0065, Japan

⁶Mechatronics Engineering Division, Politeknik Elektronika Negeri Surabaya, Indonesia

⁷Department of Electrical Engineering, Politeknik Negeri Semarang, Indonesia

E-mail: {risnumawan-anhar, achmad-aji, rochmanto-artha}@ed.tmu.ac.jp, teuku-fatahillah@ed.tmu.ac.jp, kubota@tmu.ac.jp

Assistive robots require the ability to interpret and act on natural language requests that refer to familiar places and objects, while maintaining interpretability to support user trust. Metric SLAM provides reliable geometric localization and obstacle avoidance, but it typically does not expose discrete, labeled place concepts needed for goal specification and explanation. This paper presents an online Semantic Topological Cognitive Map (STCM) that couples a LiDAR-inertial SLAM pose stream with a learned place graph and open-vocabulary semantic grounding. A dual Growing Neural Gas (GNG) architecture constructs revisitable place nodes and clusters object instances online from the robot trajectory. Open-vocabulary detections are anchored in the map frame by associating image masks with projected LiDAR points and are temporally fused into persistent node-level semantic scores. Motivated by multi-scopic neuroscience perspectives that distinguish micro-, meso-, and macroscopic levels of navigation, STCM provides a mesoscopic representation that links metric state estimation to place-based reasoning. The resulting graph supports place- and object-referenced goal selection, graph-based planning, and execution via ROS 2 Nav2, and can be queried by an LLM-based language grounding interface that maps commands to graph nodes without treating the language model as a navigation policy. Experiments on a Ranger Garden Assistant platform equipped with a 3D LiDAR and a fisheye camera demonstrate online graph construction, semantic label fusion, and autonomous navigation in indoor environments.

Keywords: Semantic topological mapping, cognitive maps, assistive navigation, Growing Neural Gas, open-vocabulary detection

1. Introduction

Robust indoor navigation requires seamless integration of adaptability and optimality across multiple temporal

and spatial scales. Adaptability depends on real-time perception and reactive control, a bottom-up learning-based approach operating at short timescales. Optimality depends on cognition and deliberative planning, a top-down knowledge-based approach operating at longer timescales. Cognitive models in neuroscience and ecological psychology can be conceptualized at micro-, meso-, and macroscopic levels, corresponding to sensing, perception, and cognition, respectively [1]. At the microscopic level, sensorimotor coordination enables immediate responses to environmental changes (e.g., obstacle avoidance during a single step). At the macroscopic level, cognitive maps support long-term planning and goal-directed behavior (e.g., route planning to a distant room). The mesoscopic level bridges these extremes, integrating bottom-up sensory evidence with top-down expectations to maintain situational awareness and medium-term adaptation (e.g., replanning a path when a previously open doorway is blocked). This multi-scopic framework suggests that robotic navigation systems should likewise integrate metric sensing, topological perception, and semantic cognition across short-, medium-, and long-term horizons, with each level informing and constraining the others.

These principles are particularly critical for assistive indoor mobile robots, which are increasingly expected to respond to natural language requests such as “go to the kitchen, find my medicine, and bring it to my bedside,” where the user specifies goals in terms of familiar places and objects rather than coordinates. In domestic settings, the world is also dynamic: objects move, doors open/close, and lighting and occlusions vary over time. In addition to reaching a goal safely, assistive navigation should be interpretable, because perceived reliability and understandable behavior influence user trust and acceptance in human-robot interaction (HRI) [2]. Fig. 1 illustrates a typical assistive scenario where a robot must understand place concepts and semantic context to fulfill natural language commands in a dynamic home environment.

A common foundation for implementing such systems is metric SLAM, which provides a pose stream and a geometric map for collision avoidance and precise planning. However, metric maps alone are insufficient for the multi-

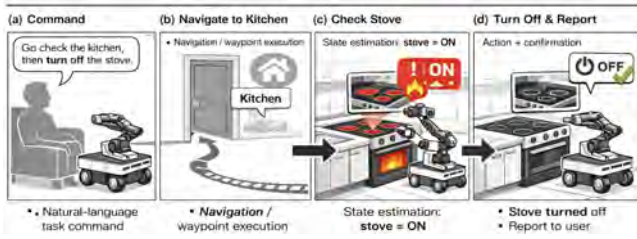


Fig. 1. Illustration of an assistive indoor navigation scenario. A robot partner must understand natural language commands that refer to familiar places (kitchen, bedside) and objects (medicine) rather than coordinates, while operating in a dynamic home environment where doors open/close, objects move, and lighting varies.

scopic integration described above: while indispensable for microscopic-level control, they usually do not encode the discrete place concepts or symbolic labels needed for macroscopic-level reasoning that aligns with how people communicate (“kitchen,” “bedside,” “doorway,” “next to the table”). The motivation for place-based abstractions is consistent with cognitive map theories in psychology and neuroscience [3, 4] and with an ecological perspective in which perception is tied to actionable environmental structure and affordances [5]. For assistive scenarios, prior work has also emphasized conversational or interaction-driven symbolic grounding for indoor environments [6] and place-oriented topological reconstruction methods for structured spaces [7].

Several lines of work attempt to bridge geometry, semantics, and planning. Hierarchical semantic mapping and dynamic scene graph systems aim to unify metric estimation with structured semantic entities. Kimera provides a framework for real-time metric–semantic localization and mapping [8], Hydra advances real-time spatial perception toward 3D scene graph construction and optimization [9], and S-Graphs+ proposes a multi-layer mapping structure with semantic and geometric layers updated online [10]. These representations can be rich, but they may emphasize object- or room-centric structure and optimization and may not be optimized for promptable, open-vocabulary place labeling or for lightweight goal grounding into a compact place graph that can serve as the primary unit of navigation and explanation.

Open-vocabulary semantic mapping approaches shift emphasis toward flexible language-like querying by leveraging foundation models. VLMaps constructs visual-language maps that allow spatial goals to be specified from free-form text, enabling navigation to concepts outside a fixed closed set [11]. ConceptFusion fuses multimodal foundation-model features into dense 3D reconstructions to support open-set semantic queries [12]. ConceptGraphs and OneMap move toward open-vocabulary 3D graph representations and real-time open-vocabulary mapping for downstream navigation tasks [13, 14]. These approaches demonstrate that open-vocabulary querying can substantially expand what a robot can be asked to find, but their representations are often dense feature fields or object-

centric graphs; consequently, they may not directly provide an online learned place-graph abstraction with temporally stabilized place labels that can be planned over, executed via standard navigation stacks, and inspected by non-expert users.

Language- and LLM-integrated systems connect instructions to plans or actions using large pre-trained models and structured scene memories. NLMap grounds LLM planning with an open-vocabulary, queryable scene representation [15]. LM-Nav combines pre-trained models with graph search for language-driven navigation [16], and GOAT accumulates an instance-aware semantic memory for long-term deployment [17]. OpenIN addresses open-vocabulary, instance-oriented navigation in dynamic domestic environments [18], and LIFGIF and MapNav integrate foundation models with more traditional navigation stacks via language-conditioned map representations [19, 20]. Many such systems, however, use custom execution back-ends and do not provide a compact ROS 2–ready place graph that can be directly inspected or edited.

Topological mapping and navigation methods focus on graph abstractions for long-horizon planning. NTS (Neural Topological SLAM) learns to build and use a topological graph for navigation [21], and ETPNav similarly performs online topological planning for vision-and-language navigation in continuous environments [22]. RoboHop proposes a segment-based topological representation that supports open-world navigation with semantic querying [23]. These approaches highlight the benefits of explicit graphs, but they may represent semantics implicitly (e.g., in latent embeddings) or may not emphasize metric anchoring and ROS 2–compatible execution with explicit symbolic place labels that can be fused and explained.

This paper targets the gap evident across the above research directions: metric SLAM is necessary for safe motion but insufficient for language-driven assistive navigation because it lacks named places and task-relevant symbolic context; meanwhile, semantics and topology are often developed as disjoint subsystems, limiting interpretability and increasing integration friction in real robots. We argue that ROS 2 / Nav2-compatible integration is not a peripheral engineering detail: assistive navigation must execute reliably on real platforms with well-tested local planning, recovery behaviors, and safety constraints, while still exposing a higher-level representation that supports human-understandable goals and explanations.

To address this gap, we develop an online Semantic Topological Cognitive Map (STCM). A LiDAR–inertial metric layer (FAST-LIO2-style) provides a map-frame pose stream [24]. A topological layer learns a place graph online from the trajectory using Growing Neural Gas (GNG) [25], with multi-scale variants for dynamic distributions [26]. Semantic grounding uses open-vocabulary detections from GroundingDINO, refined by Segment Anything [27, 28], and anchors detections into the SLAM map frame by intersecting object masks with a LiDAR point cloud that has been projected into the image

frame (per-point (u, v) alongside (x, y, z) in `lidar_link`) and then transformed to map via ROS TF. Monocular depth (Depth Anything) is used only optionally when no projected LiDAR points fall inside a detection mask [29]. Evidence is temporally fused into node-level semantic scores. Planning selects labeled nodes on the graph, while local execution uses ROS 2 Nav2 waypoints in the metric layer [30]. The STCM graph is serialized (e.g., JSON) and visualized to support debugging and explanation.

The primary contributions of this work are system and interface contributions rather than new benchmark results. Specifically, we (1) present an online STCM representation that couples metric SLAM with a semantic topological place graph for symbolic place-based planning; (2) introduce a dual-GNG learning architecture comprising a place-level GNG for revisitable place nodes and traversability edges and an instance-level GNG for persistent object instances with online outlier filtering and temporal evidence accumulation [25]; (3) develop an open-vocabulary grounding and temporal fusion pipeline that anchors detections in the map frame by associating segmentation masks with image-projected LiDAR points (with optional Depth Anything [29] fallback) and accumulates them into node-level semantic scores [27, 28]; (4) provide a ROS 2 integration that plans over graph nodes and executes via Nav2, with outputs for visualization and debugging [30]; and (5) implement an LLM-based language grounding interface that maps natural language commands to graph nodes via tool-mediated reasoning over an explicit STCM serialization, rather than using the language model as a navigation policy.

Role of the LLM is that when a large language model is used with STCM, it is not treated as an end-to-end navigation policy. The LLM is provided with the user's command and an explicit serialization of the current STCM graph (node identifiers, labels, confidence scores, and connectivity), and it outputs a goal grounded in that graph (e.g., a target node ID). This interface-based use aligns with prior work on LLMs as high-level planners when provided with explicit affordances and structured context (e.g., SayCan [31]) and with findings that structured spatial representations can reduce planning errors [32]. Goal execution, including graph search and low-level motion control, remains the responsibility of the planner and Nav2.

STCM serves as a practical middle ground between dense open-vocabulary semantic maps and full 3D dynamic scene graphs: it preserves metric mapping for reliable local control while maintaining an explicit, online-updated place graph with interpretable semantic labels suitable for assistive dialogue. Extensions toward richer hierarchical abstractions, explicit object-object relations, and systematic evaluation of assistive-task effectiveness and user trust in real homes remain future work.

2. Related Work

The motivation for place-based topological abstractions in robotics draws from cognitive map theory in psychology

and neuroscience. Tolman proposed that animals and humans develop internal spatial representations that enable goal-directed behavior beyond simple stimulus-response associations [3]. O'Keefe and Nadel provided neurobiological evidence that the hippocampus functions as a cognitive map, with place cells encoding discrete location and context information [4]. These findings suggest that spatial cognition naturally organizes around salient, stable places rather than continuous coordinate spaces, motivating graph-like representations for robot navigation.

In robotics, Kubota and collaborators have advanced the notion of informationally structured space and mutual cognitive environments for human-robot interaction. Kubota and Yorita introduced topological environment reconstruction methods that represent space as nodes (places) and edges (transitions), supporting compact navigation planning without relying solely on metric accuracy [7]. Woo and Kubota developed interactive mapping approaches that attach symbolic labels to learned places and spatial relations through human-robot conversation, emphasizing shared understanding and language grounding [6]. These works establish the principle that robot maps should support interpretable, language-ready place concepts for natural human-robot collaboration.

The ecological perspective on perception and action, introduced by Gibson, emphasizes affordances as actionable properties of the environment [5]. For assistive navigation, decision points such as doorways and junctions act as natural topological nodes, while transitions between nodes capture navigable connectivity. Labeling places by functional meaning ("kitchen," "doorway," "next to table") aligns robot spatial representations with human task descriptions and route instructions. This ecological framing motivates incremental graph learning methods that discover places online from navigation experience, such as Growing Neural Gas [25, 26].

Recent advances in metric-semantic SLAM aim to unify geometric estimation with structured semantic entities. Early foundational work by Armeni et al. introduced structured 3D scene graphs that unified semantics, 3D space, and camera viewpoints for indoor understanding [33]. Kim et al. further developed scene graph representations for intelligent agent memory, enabling robots to reason about spatial relationships between objects [34]. Kimera provides a real-time framework for metric-semantic localization and mapping, building 3D meshes with object-level semantic annotations [8]. Kimera demonstrated that detecting objects and labeling regions (rooms) during SLAM is feasible and improves the map's utility for planning and interaction [8]. However, Kimera's object detection was limited to a fixed set of categories (couches, monitors, etc.), lacking open-vocabulary flexibility, and its semantic layer did not involve language understanding for open dialogue about the map [8]. Hydra extends this toward dynamic 3D scene graphs, constructing multi-layered spatial representations (objects, rooms, buildings) and optimizing them jointly [9]. S-Graphs+ proposes a multi-layer mapping structure with semantic and geometric components updated online, supporting loop closure and global

consistency [10]. Allu et al. present a modular robotic architecture that integrates autonomous exploration with semantic map updating in large-scale indoor environments, demonstrating how modular system design can support long-duration deployment and incremental semantic refinement [35]. While these systems demonstrate that rich semantic structure can be maintained alongside metric mapping, they often emphasize dense reconstruction and may not prioritize lightweight, promptable place labeling or compact graph representations optimized for dialogue-based goal specification. STCM extends this line of work by integrating open-vocabulary grounding and providing an explicit place-graph abstraction suitable for language-based interaction.

Foundation models enable open-vocabulary semantic mapping that supports flexible language queries beyond fixed object categories. VLMaps constructs visual-language maps by associating CLIP embeddings with spatial locations, enabling navigation to goals specified by free-form text [11]. VLFM extends this concept to frontier-based exploration, using vision-language models for zero-shot semantic navigation in unknown environments [36]. ConceptFusion fuses multimodal foundation-model features into dense 3D reconstructions, supporting open-set spatial queries [12]. ConceptGraphs and OneMap advance open-vocabulary 3D scene graph representations and real-time mapping for downstream navigation tasks [13, 14]. Semantic exploration approaches such as SemExp demonstrate how semantic knowledge can guide active exploration and object goal navigation [37]. While these approaches substantially expand the range of queryable concepts, their representations are often dense feature fields or object-centric graphs. They may not directly provide an online-learned, temporally stabilized place-graph abstraction with explicit symbolic labels that can be inspected, planned over, and executed via standard navigation stacks.

Large language models and pre-trained vision-language models enable instruction-driven navigation. NLMap grounds LLM-generated plans with an open-vocabulary, queryable scene representation [15]. LM-Nav combines pre-trained models with graph search for language-conditioned navigation [16], and GOAT accumulates instance-aware semantic memory for long-term deployment in dynamic environments [17]. OpenIN addresses open-vocabulary, instance-oriented navigation in domestic settings [18], while LIFGIF and MapNav integrate foundation models with navigation stacks via language-conditioned map representations [19, 20]. SORT3D introduces a spatial object-centric reasoning toolbox that leverages LLMs for zero-shot 3D grounding, providing structured spatial reasoning capabilities over object-centric scene representations [38]. Many such systems use custom execution back-ends and do not provide a compact, ROS 2-compatible place graph that can be directly inspected or edited. Furthermore, the role of LLMs in these systems is often end-to-end, potentially obscuring the explicit grounding of goals in a structured spatial representation.

Explicit graph-based navigation methods focus on topo-

logical abstractions for long-horizon planning. Neural Topological SLAM (NTS) learns to build and use a topological graph for navigation in novel environments [21]. ETPNav performs online topological planning for vision-and-language navigation in continuous spaces [22]. RoboHop proposes a segment-based topological representation supporting open-world navigation with semantic querying [23]. These approaches highlight the value of explicit graphs for interpretability and planning efficiency, but they may represent semantics implicitly in latent embeddings or may not emphasize metric anchoring, temporal fusion of open-vocabulary detections, and direct integration with ROS 2 navigation interfaces.

Biological cognitive systems operate across multiple temporal and spatial scales, seamlessly integrating short-term reactive behaviors with long-term deliberative planning. Saputra et al. propose a multi-scopic neuro-cognitive framework for legged locomotion that explicitly organizes intelligent functions at micro-, meso-, and macro-scopic levels [1]. The microscopic level corresponds to sensing and short-term adaptation, handling immediate sensorimotor coordination (e.g., adjusting leg swing to avoid a sudden obstacle detected mid-step). The macroscopic level corresponds to cognition and long-term adaptation, managing knowledge building and goal-directed behavior planning (e.g., selecting an optimal route through a previously mapped environment). The mesoscopic level bridges these extremes, integrating bottom-up perceptual information with top-down map knowledge to support medium-term adaptation (e.g., localizing within a topological map and dynamically replanning when a planned path becomes obstructed). This conceptualization provides a principled way to manage computational complexity: low-level processes run at high frequencies with local scope, while high-level processes operate at lower frequencies over global structures, with intermediate processes mediating information flow between levels. Experimental results on quadruped robots demonstrated that this multi-scopic integration achieves both adaptability to unexpected terrain and optimality in path efficiency, with stable memory usage because environmental reconstruction at the mesoscopic level can be maintained as temporal working memory while only the compact cognitive map persists as long-term memory [1].

Kubota et al. extend the multi-scopic philosophy to cyber-physical systems, introducing multi-scopic topological twins that support hierarchical organization of topological submaps and scalable planning across viewpoints [39]. These works reinforce the need for multi-scale, updateable representations that can adapt to dynamic environments and support both local detail and global structure. STCM aligns with this philosophy by maintaining a dual representation of metric for local control and topological for task-level planning, organized conceptually along micro-, meso-, and macro-scopic lines. STCM is compatible with future extensions toward hierarchical place abstractions (e.g., room-level, floor-level, and building-level graphs) and explicit coupling of perception-action cycles at multiple timescales.

The present work targets the integration gap evident across the above research directions: metric SLAM provides essential geometric grounding but lacks named places and task-relevant symbolic context; meanwhile, semantics and topology are often developed as separate subsystems or as dense representations not optimized for dialogue-based assistive navigation. We develop STCM to bridge this gap: an online place graph learned from SLAM trajectories via GNG, enriched with open-vocabulary semantic labels temporally fused into node attributes, and directly compatible with ROS 2 Nav2 for reliable execution. This representation provides an explicit, interpretable interface for language-based goal specification, planning transparency, and assistive human-robot interaction.

3. Methodology

3.1. System Architecture and Design Philosophy

The STCM system integrates modern ROS 2 navigation infrastructure with topological learning and semantic labeling for assistive navigation tasks. The architecture maintains a dual representation: a continuous metric map for precise local control and obstacle avoidance, and a discrete topological place graph for task-level planning, semantic grounding, and interpretable communication with users.

Conceptually, STCM embodies a multi-scope neuro-cognitive architecture that organizes intelligent functions across three interconnected levels. At the **microscopic level**, real-time metric SLAM and reactive semantic detection provide immediate sensorimotor feedback for short-term adaptation (e.g., continuous pose tracking, frame-by-frame object detection). At the **macroscopic level**, the cognitive map and graph-based path planner support long-term deliberative behavior (e.g., global route planning, persistent place labeling). Bridging these levels, the **mesoscopic level** maintains topological structure learning and localization, integrating bottom-up perception (incoming sensor observations) with top-down knowledge (existing map structure) to enable medium-term adaptation such as online node creation, edge updates, and dynamic replanning when environmental changes invalidate the current plan. This multi-scope organization mirrors cognitive architectures in biological systems, where perception, action, and cognition operate at distinct but coupled timescales [1], and ensures that STCM can respond both reactively to immediate obstacles and deliberately to high-level navigation goals.

The core components operate in a pipeline architecture. Sensor inputs from a 176° field-of-view (FOV) fisheye camera and Livox Mid-360 3D LiDAR are fused with inertial measurements. A LiDAR-inertial odometry (LIO) module based on FAST-LIO2 provides real-time pose estimation $\mathbf{x}_t$ and constructs a metric map $\mathcal{M}$ via iterative least-squares minimization of LiDAR-inertial residuals [24]. The robot's 2D position in the map frame $\mathbf{y}_t = (x_t, y_t)$ is extracted from the 6-DOF pose and serves as in-

put to the topological learning layer.

The topological layer employs a dual Growing Neural Gas (GNG) architecture [25, 26]. First, place-level GNG incrementally learns place nodes from the streaming pose observations. At each time step, GNG associates the current position $\mathbf{y}_t$ with the nearest existing place prototype $\mathbf{r}_i$ by computing $i^* = \arg \min_i \|\mathbf{y}_t - \mathbf{r}_i\|$. When the robot is far from all existing nodes (distance exceeding threshold D_{new}), a new place node is created. Edges between nodes are added when the robot traverses from one node to another, encoding observed navigability and connectivity. Second, instance-level GNG operates independently on detected object positions to cluster multiple instances of the same semantic class, enabling the system to distinguish individual objects (e.g., three distinct chairs) while filtering spurious detections through adaptive prototype learning.

Semantic grounding is achieved through open-vocabulary object detection using GroundingDINO, optionally refined by the Segment Anything Model (SAM) [27, 28]. Detected objects are anchored in the map frame primarily by intersecting detection masks with a LiDAR point cloud that has been projected into the image frame, producing a set of labeled 3D positions $\mathcal{O}_t = \{(l_i, \mathbf{p}_i^{\text{map}}, c_i)\}_i$ where l_i is the class label, $\mathbf{p}_i^{\text{map}}$ is the map-frame position, and c_i is the detection confidence. Monocular depth (Depth Anything) is supported only as an optional fallback when the projected LiDAR provides no points on an object [29]. Instance-level GNG clusters these detections into stable object instances, which are then associated with place nodes for temporal fusion into semantic score vectors $\mathbf{s}_i$. The dominant class label $\ell_i = \arg \max_c s_{i,c}$ becomes the interpretable place label.

The resulting STCM graph $G = (V, E)$ consists of nodes V with attributes $\{\mathbf{r}_i, \mathbf{s}_i, \ell_i\}$ and edges E representing traversable connections. Navigation proceeds in two stages: high-level graph planning selects a sequence of place nodes based on semantic goals (e.g., “go to kitchen”), and low-level execution uses Nav2 to follow metric waypoints corresponding to the selected nodes [30]. This architecture enables both symbolic reasoning over place concepts and reactive, collision-free control.

Fig. 2 illustrates the complete STCM pipeline architecture, showing the flow from sensor inputs through metric SLAM, semantic grounding, topological learning, and navigation execution.

3.2. Metric Layer: LiDAR-Inertial Odometry and Mapping

The metric layer employs FAST-LIO2, a computationally efficient LiDAR-inertial odometry system that tightly couples 3D LiDAR measurements with inertial data [24]. Reported FAST-LIO2 results indicate localization errors on the order of 5–6 cm in long-duration runs [24], providing a robust metric foundation for STCM's topological and semantic layers. The system formulates odometry as an iterative Extended Kalman Filter (iEKF) that estimates the robot state $\mathbf{x}_t = [\mathbf{p}_t, \mathbf{v}_t, \mathbf{q}_t, \mathbf{b}_a, \mathbf{b}_g]$, where $\mathbf{p}_t$ is position, $\mathbf{v}_t$ is velocity, $\mathbf{q}_t$ is orientation (quaternion), and $\mathbf{b}_a, \mathbf{b}_g$ are

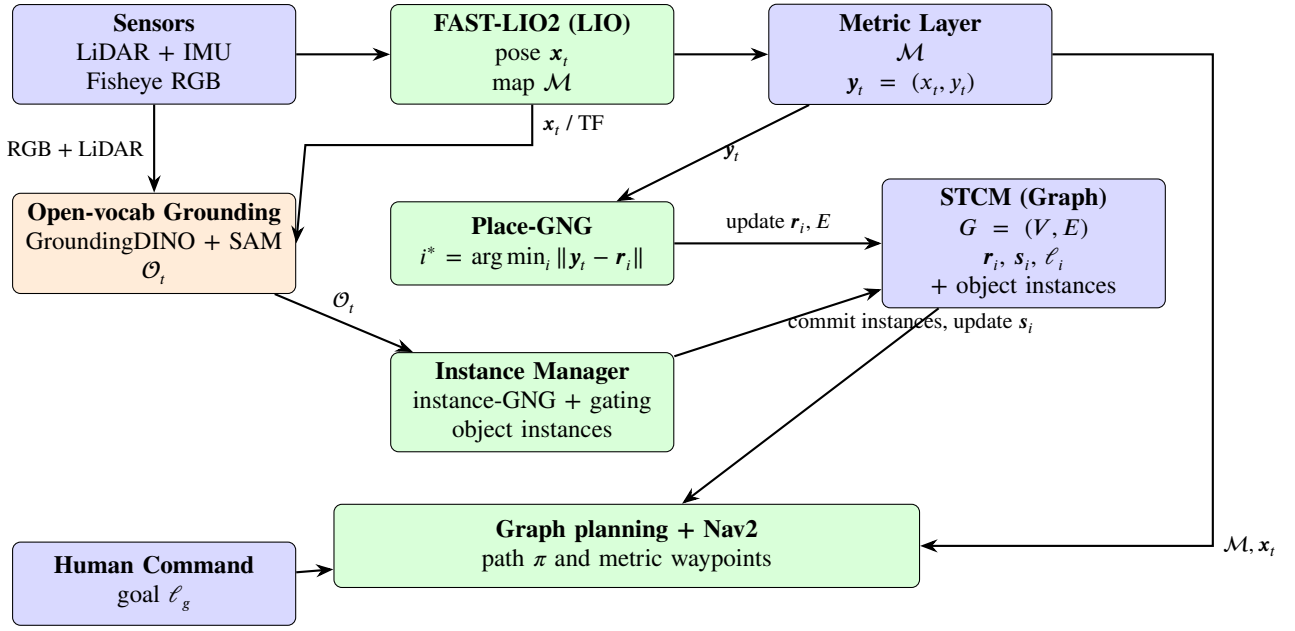


Fig. 2. STCM system pipeline architecture. The pipeline integrates metric SLAM (FAST-LIO2), open-vocabulary grounding (GroundingDINO + SAM), dual-GNG learning (place-GNG for revisitable place nodes and an instance-GNG manager for persistent object instances), and ROS 2 Nav2 execution. Sensor inputs flow through LiDAR–inertial odometry to produce a metric map and pose stream, while open-vocabulary detections are grounded into the map frame and managed as object instances. Place–object associations temporally fuse semantic evidence into node-level score vectors, yielding an STCM graph that supports language-based goal specification, graph planning, and metric waypoint execution.

accelerometer and gyroscope biases. The system maintains an incremental k-d tree (ikd-Tree) representation of the map $\mathcal{M}$ for efficient nearest-neighbor queries during scan matching.

At each iteration, IMU measurements propagate the state forward via the standard kinematic model. LiDAR points are then registered to the map by solving a point-to-plane iterative closest point (ICP) problem. The measurement residual for point j in scan t is defined as in Eq. (1):

$$r_j = (T_t p_j^{\text{lidar}} - p_j^{\text{map}}) \cdot n_j, \quad \dots \quad (1)$$

where T_t is the pose transform, p_j^{lidar} is the point in the LiDAR frame, p_j^{map} is the corresponding point in the map, and n_j is the local surface normal. The state is updated by minimizing the weighted sum of squared residuals via Gauss-Newton iterations within the EKF update step.

The output of FAST-LIO2 is a high-rate pose stream (typically 10–20 Hz) and a globally consistent point cloud map $\mathcal{M}$. For STCM, we extract the planar components (x_t, y_t) of the estimated position p_t and denote the resulting 2D position stream as $y_t := [x_t, y_t]^\top$. This stream drives the topological learning module and provides the frame for anchoring semantic detections.

3.3. Topological Layer (place-GNG) for Place Discovery

Growing Neural Gas is an unsupervised incremental clustering algorithm that adapts its topology and number of nodes online in response to input data [25]. GNG provides

an effective balance between dense metric maps and sparse landmark representations, being computationally light yet informative for navigation [40]. We employ GNG to convert the continuous trajectory $\{y_t\}$ from SLAM into a discrete place graph suitable for symbolic reasoning and planning. The topological abstraction enables efficient graph-level planning with far fewer nodes than grid cells, making computation tractable without losing essential connectivity information [40]. Fig. 3 illustrates the place-GNG mechanism as the robot traverses a corridor, showing node insertion, adaptation, edge creation, and semantic association.

3.3.1. Node Association and Adaptation

At each time step t , the current robot position y_t is presented to the GNG network. The algorithm computes distances to all existing nodes as in Eq. (2):

$$d_i = \|y_t - r_i\|, \quad \dots \quad (2)$$

and identifies the winner (nearest node) and second-nearest node:

$$i^* = \arg \min_i d_i, \quad i_2 = \arg \min_{i \neq i^*} d_i. \quad \dots \quad (3)$$

The winner node’s prototype is adapted toward the input using a learning rate ϵ_b as in Eq. (4):

$$r_{i^*} \leftarrow r_{i^*} + \epsilon_b (y_t - r_{i^*}). \quad \dots \quad (4)$$

Topological neighbors of i^* (nodes connected by edges) are also adapted with a smaller learning rate ϵ_n as in Eq. (5):

$$r_n \leftarrow r_n + \epsilon_n (y_t - r_n), \quad \forall n \in \mathcal{N}(i^*), \quad \dots \quad (5)$$

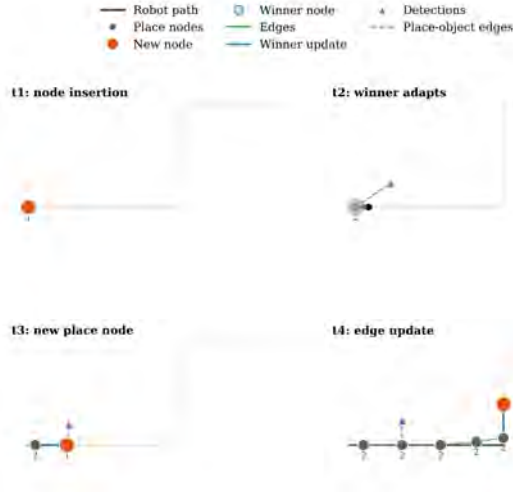


Fig. 3. Place-level GNG illustration. As the robot traverses the corridor, new nodes are inserted when the current pose is farther than D_{new} from existing prototypes, and nearby nodes adapt toward revisited poses. Edges encode observed transitions between successive winners. Purple triangles represent object detections, and dashed purple links show the temporary place-object associations used for semantic fusion.

where $\mathcal{N}(i^*)$ denotes the set of topological neighbors. This local adaptation ensures that place prototypes remain representative of the regions where the robot spends time.

3.3.2. Edge Creation and Aging

Connectivity in the place graph is established through edge creation and aging mechanisms. When the winner i^* is determined, an edge is created or refreshed between i^* and the second-nearest node i_2 as in Eq. (6):

$$\text{if } (i^*, i_2) \notin E, \text{ then } E \leftarrow E \cup \{(i^*, i_2)\}. \quad \dots \quad (6)$$

The age of this edge is reset to zero. All edges emanating from i^* have their ages incremented. Edges whose ages exceed a threshold a_{max} are removed, allowing the graph to forget transient connections that no longer reflect current traversability.

For STCM, we track the previous winner node i_{t-1} . When the winner changes (i.e., $i_{t-1} \neq i^*$), we add or refresh the edge (i_{t-1}, i^*) . This ensures that edges represent observed robot transitions between places, capturing navigable connectivity rather than arbitrary proximity. Fig. 3 shows these edges encoding the observed transitions between successive winner nodes.

3.3.3. Node Insertion

If the robot moves to a region not well represented by existing nodes, a new place node is created. The insertion criterion is given in Eq. (7):

$$\text{if } d_{i^*} > D_{\text{new}}, \text{ add node } v_{\text{new}} : \mathbf{r}_{\text{new}} \leftarrow \mathbf{y}_t. \quad \dots \quad (7)$$

The threshold D_{new} controls the granularity of the place graph. Smaller values produce finer-grained representations with more nodes, while larger values yield coarser

abstractions. For indoor navigation, we set $D_{\text{new}} = 0.5$ m to capture fine-grained topological structure. As illustrated in Fig. 3, new nodes are inserted when the robot enters unexplored regions farther than D_{new} from existing prototypes.

Alternatively, standard GNG uses an error-driven insertion strategy: nodes accumulate error proportional to their distance to inputs, and periodically the node with maximum error spawns a new node between itself and its neighbor with the highest error. For real-time mapping, we favor the distance-threshold criterion for predictable memory usage and computational cost.

3.4. Semantic Grounding Pipeline

The semantic grounding pipeline transforms raw RGB images and LiDAR measurements into labeled 3D object anchors that are fused into the STCM graph. The pipeline consists of five stages: open-vocabulary detection, instance segmentation, metric grounding using image-projected LiDAR (with optional monocular depth fallback), transformation to the map frame, and fusion into place nodes.

3.4.1. Stage 1: Open-Vocabulary Detection

GroundingDINO is a transformer-based detector that conditions on text prompts to detect objects in arbitrary categories without task-specific fine-tuning [27]. The model achieves strong zero-shot detection performance (52.5 AP on COCO zero-shot benchmarks [27]), enabling STCM to localize objects of any category given a label in natural language. This open-vocabulary capability allows the robot to ground semantic labels such as “exit sign,” “medicine cabinet,” or “bedside table” even if these categories were not predefined during system development. Given an RGB image I_{rgb} and a set of text prompts $\mathcal{T}$ (e.g., “door . table . sink . stove”), GroundingDINO outputs a set of bounding boxes $\{b_i\}$, class labels $\{l_i\}$, and confidence scores $\{c_i\}$ for detected objects (Fig. 4). Detections with confidence below threshold τ_{box} and text-matching scores below τ_{text} are filtered.

3.4.2. Stage 2: Instance Segmentation Refinement

To improve the spatial precision of semantic evidence, we refine bounding boxes with pixel-level instance masks using the Segment Anything Model (SAM) or its lightweight variant MobileSAM [28]. SAM’s zero-shot ability to produce accurate masks for novel object types [28] enables the robot to isolate arbitrary landmarks (e.g., unusual furniture, decorative objects) on the map without requiring category-specific training. Each bounding box b_i serves as a prompt to SAM, which produces a binary mask M_i delineating the object’s spatial extent. Masks suppress background clutter and improve the accuracy of associating image-projected LiDAR points to the object region.

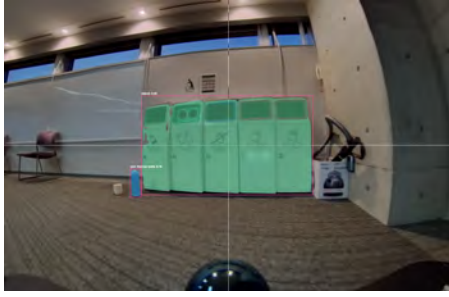


Fig. 4. Object detection and segmentation pipeline. GroundingDINO detects objects from text prompts (e.g., “bottle,” “trash bins”) and produces bounding boxes, which SAM refines into pixel-level masks. Segmentation constrains the association of image-projected LiDAR points to the object region, improving 3D anchoring accuracy.

3.4.3. Stage 3: Metric Grounding via Image-Projected 3D LiDAR (with Optional Depth Fallback)

For each detected object i , we estimate its 3D position by intersecting the detection mask M_i with a LiDAR point cloud that has been projected into the image frame. At time t , the robot publishes a point cloud message in which each point carries both its 3D coordinates $\mathbf{p}_j^{\text{lidar}} = (x_j, y_j, z_j)$ expressed in `lidar_link` and its corresponding image coordinates $\mathbf{u}_j = (u_j, v_j)$ (pixels). We select the subset of LiDAR points whose projections fall inside the segmentation mask as in Eq. (8):

$$\mathcal{P}_{i,t} = \{\mathbf{p}_j^{\text{lidar}} : \mathbf{u}_j \in M_i\}. \quad (8)$$

If $\mathcal{P}_{i,t}$ is nonempty, we estimate the object anchor in `lidar_link` by choosing the most frequent 3D point among the associated LiDAR returns, and denote it $\mathbf{p}_i^{\text{lidar}}$. To compute the most frequent point from the continuous-valued set $\mathcal{P}_{i,t}$, we discretize the 3D space via voxelization with a fixed bin size and select the centroid of the bin containing the largest number of points.

When no projected LiDAR points fall inside M_i (e.g., due to sparsity or occlusion), we optionally use a monocular depth estimator (Depth Anything [29]) to obtain a dense depth map and back-project pixels in M_i to a 3D estimate. This depth fallback was not employed in the experiments reported in this paper. The fallback mechanism is thus not required for the core STCM pipeline.

3.4.4. Stage 4: Transformation to Map Frame

To integrate semantic evidence across viewpoints and robot motion, we transform object positions to the global map frame. Because the metric anchor $\mathbf{p}_i^{\text{lidar}}$ is expressed in `lidar_link`, we use the ROS 2 TF (transform) tree to obtain the transform $T_{\text{lidar}}^{\text{map}}$ at time t . In our setup, the relevant chain is `map`→`odom`→`base_footprint`→`base_link`→`lidar_link`. For each object, the map-frame anchor is computed as in Eq. (9):

$$\mathbf{p}_i^{\text{map}} = T_{\text{lidar}}^{\text{map}} \begin{bmatrix} \mathbf{p}_i^{\text{lidar}} \\ 1 \end{bmatrix}. \quad (9)$$

The output is $\mathcal{O}_t = \{(l_i, \mathbf{p}_i^{\text{map}}, c_i)\}_i$, a set of globally anchored semantic observations.

3.4.5. Stage 5: Fusion into STCM Place Graph

Semantic evidence is accumulated into the place nodes identified by GNG. At time t , the winner node i^* has been determined from the robot’s position. We construct a semantic observation vector $\mathbf{o}_t \in \mathbb{R}^{|C|}$ where each component $o_{t,c}$ aggregates confidence for class c from detections in $\mathcal{O}_t$. In our default deployment, we use max aggregation: when multiple detections with label c are present at time t , we set $o_{t,c} = \max_{j:l_j=c} c_j$, i.e., the maximum confidence among all detections of class c in $\mathcal{O}_t$. If no detection with label c is present, $o_{t,c} = 0$. (The system also supports sum aggregation via the `place_gng_semantic_aggregation` parameter, but max is the default.)

The node’s semantic score vector $\mathbf{s}_{i^*}$ is updated using exponential smoothing as in Eq. (10):

$$\mathbf{s}_{i^*} \leftarrow (1 - \alpha) \mathbf{s}_{i^*} + \alpha \mathbf{o}_t, \quad (10)$$

where $\alpha \in [0, 1]$ is the temporal fusion rate. This accumulation stabilizes semantic labels against frame-to-frame detection noise. The place label is assigned as the class with maximum accumulated score as in Eq. (11):

$$\ell_{i^*} = \arg \max_c \mathbf{s}_{i^*,c}. \quad (11)$$

Optionally, we can also store the metric anchors $\{\mathbf{p}_i^{\text{map}}\}$ as object-level landmarks associated with the place node, enabling finer-grained spatial queries and affordance reasoning.

3.5. Instance-Level GNG for Semantic Object Management

While place-level GNG (Section 3.3) discovers topological nodes from robot trajectories, the semantic layer must also distinguish multiple instances of the same object class. For example, a corridor may contain multiple chairs, tables, or doors. Simple distance-based merging (checking if a new detection is within a fixed radius of existing detections) suffers from two problems: (1) threshold sensitivity, where too large a threshold causes distant instances to merge incorrectly and too small a threshold causes the same object to create duplicates under localization drift; (2) lack of adaptability, since static thresholds cannot handle object scale variation or evolving spatial distributions.

To address this, we employ a second GNG module operating at the instance level: for each semantic class $c \in C$, we maintain a separate incremental Growing Neural Gas model GNG_c that clusters observed object positions into distinct instance prototypes. This instance-level GNG provides adaptive, online clustering that learns the spatial distribution of each object class and naturally handles revisits, localization uncertainty, and scale variation.

3.5.1. Per-Label Instance Clustering

For each semantic class c , we maintain a GNG model $\text{GNG}_c = (\mathcal{I}_c, \mathcal{E}_c)$, where $\mathcal{I}_c$ is the set of instance prototype

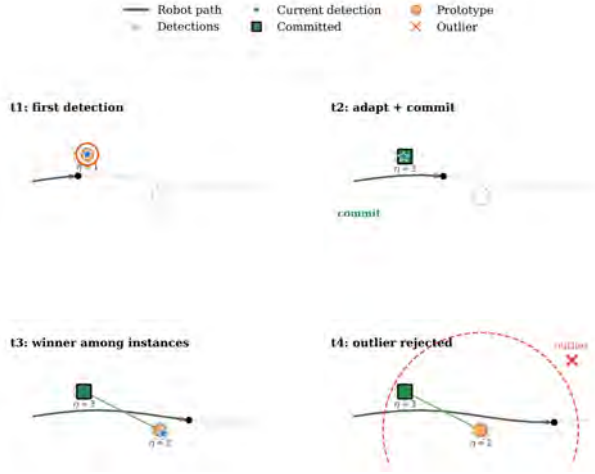


Fig. 5. Instance-level GNG illustration. As the robot moves, repeated detections of the same object arrive with small positional variation. The winner prototype adapts toward consistent observations, while detections that fall beyond the gating distance are rejected as outliers. This sequential filtering yields stable instance prototypes despite noise and revisits.

nodes and $\mathcal{E}_c$ is the set of instance association edges. Each instance node $\iota_j \in \mathcal{I}_c$ is characterized by three attributes: a prototype position $\mathbf{m}_j \in \mathbb{R}^3$ representing the 3D location in the map frame, an observation count η_j tracking the number of detections associated with this instance, and an accumulated confidence ϕ_j that sums the detection confidences over all observations.

When a new detection $(l, \mathbf{p}^{\text{map}}, \gamma)$ arrives with label $l = c$ and confidence γ , we present its map-frame position $\mathbf{p}^{\text{map}}$ to GNG_c . The GNG algorithm proceeds as follows (illustrated in Fig. 5):

Step 1: Outlier gating. To prevent spurious detections from corrupting the instance set, we first check if the new position is within a gate distance D_{gate} of any existing instance as in Eq. (12):

$$d_{\min} = \min_j \|\mathbf{p}^{\text{map}} - \mathbf{m}_j\|. \quad (12)$$

If $d_{\min} > D_{\text{gate}}$ and $|\mathcal{I}_c| > 0$, the detection is classified as an outlier and discarded. This prevents false positives and distant misdetections from creating spurious instances.

Step 2: Winner selection. Compute distances to all instance prototypes as in Eq. (13):

$$d_j = \|\mathbf{p}^{\text{map}} - \mathbf{m}_j\|, \quad j \in \mathcal{I}_c, \quad (13)$$

and identify the nearest (winner) and second-nearest instances:

$$j^* = \arg \min_j d_j, \quad j_2 = \arg \min_{j \neq j^*} d_j. \quad (14)$$

Step 3: Prototype adaptation. The winner prototype is adapted toward the new observation as in Eq. (15):

$$\mathbf{m}_{j^*} \leftarrow \mathbf{m}_{j^*} + \epsilon_w (\mathbf{p}^{\text{map}} - \mathbf{m}_{j^*}), \quad (15)$$

where ϵ_w is the winner learning rate. Topological neighbors of j^* (instances connected by edges in $\mathcal{E}_c$) are adapted

with a smaller rate ϵ_n^{inst} as in Eq. (16):

$$\mathbf{m}_k \leftarrow \mathbf{m}_k + \epsilon_n^{\text{inst}} (\mathbf{p}^{\text{map}} - \mathbf{m}_k), \quad \forall k \in \mathcal{N}_c(j^*), \quad (16)$$

where $\mathcal{N}_c(j^*)$ denotes the topological neighbors in $\mathcal{E}_c$.

Step 4: Edge creation and aging. An edge between j^* and j_2 is created or refreshed as in Eq. (17):

$$\text{if } (j^*, j_2) \notin \mathcal{E}_c, \text{ then } \mathcal{E}_c \leftarrow \mathcal{E}_c \cup \{(j^*, j_2)\}. \quad (17)$$

Edge ages are incremented, and edges exceeding age threshold $a_{\max}^{\text{inst}}$ are removed.

Step 5: Node insertion. If the winner distance exceeds a creation threshold $D_{\text{new}}^{\text{inst}}$, a new instance node is created as in Eq. (18):

$$\text{if } d_{j^*} > D_{\text{new}}^{\text{inst}}, \text{ create } \iota_{\text{new}} : \quad \mathbf{m}_{\text{new}} \leftarrow \mathbf{p}^{\text{map}}, \\ \eta_{\text{new}} = 1, \quad \phi_{\text{new}} = \gamma. \quad (18)$$

Otherwise, the winner's observation count and confidence are updated:

$$\eta_{j^*} \leftarrow \eta_{j^*} + 1, \quad \phi_{j^*} \leftarrow \phi_{j^*} + \gamma. \quad (19)$$

Step 6: Commitment to semantic graph. Instance nodes are committed to the STCM semantic graph only after accumulating a minimum number of observations $\eta_{\min}$ (typically 3–5). This filters transient detections and ensures that graph nodes represent stable, repeatedly observed objects. When $\eta_j \geq \eta_{\min}$ and the instance is not yet in the graph, a new graph node is created with label c , position $\mathbf{m}_j$, and initial confidence ϕ_j/η_j .

3.5.2. Integration with Place Graph

The instance-level GNG operates independently from the place-level GNG but shares the same map frame. Instance prototypes $\{\mathbf{m}_j\}$ represent distinct physical objects, while place nodes $\{\mathbf{r}_i\}$ represent navigable locations. During semantic fusion (Section 3.4.5), when the robot visits place node i^* , the semantic observation vector $\mathbf{o}_i$ is constructed by aggregating confidences from all instances of each class within a visibility radius of the current pose. This dual-layer representation enables both discrete object-level reasoning (“there are three chairs in the corridor”) and continuous place-level reasoning (“the table area has high scores for table and chair”).

3.6. STCM Graph Representation and Output

The STCM graph $G = (V, E)$ is the central data structure for task planning and human-robot communication. The complete formalization is given in Eq. (20):

$$G = (V, E),$$

$$v_i \in V : \left\{ \begin{array}{l} \mathbf{r}_i \in \mathbb{R}^2, \\ \mathbf{s}_i \in \mathbb{R}^{|C|}, \\ n_i \in \mathbb{N}, \\ \ell_i = \arg \max_{c \in C} s_{i,c} \end{array} \right\}, \quad (20)$$

where C is the set of semantic classes, $\mathbf{r}_i$ is the place position, $\mathbf{s}_i$ is the semantic score vector, n_i is the visit count, and ℓ_i is the assigned place label.

Each node $v_i \in V$ represents a place characterized by four principal attributes. The **position** $r_i \in \mathbb{R}^2$ encodes the representative 2D coordinates of the place in the map frame, serving as the navigational target. The **semantic scores** $s_i \in \mathbb{R}^{|C|}$ store confidence values for each semantic class in C , reflecting the evidence accumulated from visual observations during traversal. The **label** ℓ_i denotes the dominant semantic class assigned to the place (e.g., “kitchen” or “doorway”), determined by the highest-scoring element of s_i . Finally, the **visit count** n_i records the number of times the node was selected as the winner during GNG updates, providing a measure of node stability and observational support.

Each edge $e_{ij} \in E$ represents an observed transition between places, with optional attributes such as traversal count or traversal cost.

The STCM graph provides outputs for planning, inspection, and explainability. We export the graph as a NetworkX graph object, as a JSON file (`stcm.json`), and as ROS 2 topics (`/semantic_graph/nodes`, `/semantic_graph/place_graph`) for real-time visualization in RViz. Each node contains `id` (unique identifier), `label` (semantic class), `pose` (x, y coordinates), `scores` (confidence values for all $|C|$ semantic classes), and `visits` (observation count). Each edge contains `source` and `target` node IDs, `traversals` (transition count), and `age` (edge age for pruning). Exposing the node scores vector provides the evidence behind labels (e.g., high scores for “sink” and “stove” justify a “kitchen” label), enabling transparent reasoning about place assignments.

The graph is serialized to JSON for storage, visualization, and interfacing with external systems. A sample node in JSON format:

```
{
  "id": "place_12",
  "label": "kitchen",
  "pose": [2.34, -1.12],
  "scores": {"sink": 0.71, "stove": 0.63,
            "table": 0.22, ...},
  "visits": 47
}
```

Note: The `scores` field contains confidence values for all configured semantic classes; only non-zero scores are shown above for brevity.

For deployment operation, the generated JSON is loaded by the ROS stacks and read by the LLM.

3.7. Navigation and Planning

Navigation in STCM operates at two levels. When a user issues a command such as “go to the kitchen,” the system first grounds the goal in the STCM graph by identifying nodes whose labels match the command. A graph search algorithm (e.g., Dijkstra or A*) computes a path $\pi = [v_1, v_2, \dots, v_k]$ through the place graph. Topological planning is computationally efficient: planning over tens of graph nodes is near-instantaneous compared to searching large metric spaces with hundreds of thousands of

poses [21]. Graph-based planning time grows roughly with the number of places rather than environment area, which is favorable for scalability in large buildings.

The positions $\{r_i\}$ of nodes along the path are then sent as waypoints to the ROS 2 Nav2 stack [30]. Nav2 performs metric planning on a global costmap constructed from $\mathcal{M}$ and executes local obstacle avoidance using its local planner/controller. In our configuration (Section 4.3), Nav2 uses the Smac Planner 2D for global planning and the MPPI (Model Predictive Path Integral) controller for local trajectory optimization. The MPPI controller generates velocity commands (v_x, v_y, ω) suitable for omnidirectional bases by sampling stochastic trajectories and optimizing over a receding-horizon cost function that balances path-following, goal-reaching, and obstacle avoidance while respecting kinematic constraints. Recovery behaviors are triggered if progress stalls, and the robot can re-plan at the topological level if a place becomes unreachable. This hierarchical approach has been shown to improve efficiency and robustness in long-horizon navigation tasks [21], as agents can select intermediate targets on the graph rather than operating purely on low-level sensory inputs.

This two-tier architecture ensures that high-level goals are interpretable and grounded in symbolic place concepts, while low-level execution remains robust and compatible with standard navigation infrastructure. The STCM graph nodes can incorporate semantic information (e.g., `node = “Elevator Lobby”`), making the plan explainable in human terms such as “take the elevator to floor 3, then navigate to the kitchen” [30].

3.8. LLM-Based Language Grounding and Planning

For complex natural language instructions that involve spatial reasoning (e.g., “go to the table between the two chairs” or “navigate to the smallest chair near the door”), we extend STCM with an LLM-based planning interface built on LangChain and LangGraph. The LLM serves as a high-level planner that grounds language commands in the STCM object graph and generates navigation sequences, while graph search and Nav2 remain responsible for execution.

The system provides the LLM with a structured representation of the environment: a list of detected objects from the STCM graph, each with an identifier, semantic label, and map-frame coordinates. The LLM is equipped with a toolbox of spatial reasoning functions including `find_near(target, anchor)`, `find_between(target, anchor1, anchor2)`, `find_left/right/above/below(target, anchor)`, and ordering tools such as `order_left_to_right(target)` and `order_smallest_to_largest(target)`. These tools implement geometric predicates using object centroids, bounding boxes, and viewpoint-relative spatial relations, providing symbolic abstractions over the metric map.

Given a natural language command, the LLM generates a sequence of tool calls to re-

solve spatial references, followed by a final `command_robot(list_of_commands)` call specifying navigation primitives: `go_near(object_id)` or `go_between(object_id1, object_id2)`. For example, the instruction “go to the second chair from the left near the table” is decomposed into: (1) `find_near('`chair'`, '`table'')` to identify chairs near tables, (2) `order_left_to_right('`chair'')` to rank them spatially, (3) select the second ID, and (4) `command_robot(['`go_near'`, [id]])`. An optional actor-critic architecture allows a second LLM to review the reasoning trace and provide corrective feedback before execution.

The output navigation commands reference object IDs in the STCM graph. These are resolved to metric waypoints via the object centroids $\{p_i^{\text{map}}\}$ or place node positions $\{r_i\}$, which are then executed by Nav2. This design aligns with recent findings that providing LLMs with explicit graph structure improves spatial reasoning and reduces planning errors [32], while keeping metric planning and control separate ensures safety and ROS 2 compatibility. The LLM interface enables flexible language-driven interaction without requiring end-to-end learning or custom low-level controllers.

4. Implementation and Experimental Setup

4.1. Robot Platform

We have developed the Ranger Garden Assistant mobile robot platform as an integrated system for assistive navigation research. The complete ROS 2 software stack, including hardware integration packages, sensor drivers, SLAM configurations, and navigation parameter files, is made publicly available to support reproducibility and future research at <https://github.com/anh0001/ranger-garden-assistant>. We implement and evaluate STCM on this platform, shown in Fig. 6. The platform is built upon an AgileX Ranger Mini 3.0 omnidirectional base equipped with sensing and manipulation hardware for outdoor and indoor assistive tasks. The robot’s computing platform runs Ubuntu 22.04 with ROS 2 Humble, providing the middleware foundation for sensor integration, navigation, and system coordination.

The mobile base employs an AgileX Ranger Mini 3.0 omnidirectional drive system with swivel wheels on each corner, enabling lateral motion and in-place rotation essential for maneuvering in constrained indoor spaces. This omnidirectional capability allows the robot to adjust its orientation and position independently, facilitating navigation in narrow corridors and doorways typical of domestic environments. The base supports a maximum linear velocity of 0.26 m/s and angular velocity of 1.0 rad/s, values selected to balance navigation efficiency with safety in human-occupied spaces.

For 3D perception, the platform integrates a Livox Mid-360 3D LiDAR sensor with 360° horizontal field of view, detection range up to 70 m, and point rate of 200,000 points



Fig. 6. Ranger Garden Assistant mobile robot platform. The system integrates an AgileX Ranger Mini 3.0 omnidirectional base, Livox Mid-360 3D LiDAR, Tier IV C2-176 fisheye camera (176° FOV), and AgileX PiPER 6-DOF robotic arm, running ROS 2 Humble with Navigation2 and FAST-LIO2.

per second. The Mid-360 employs a non-repetitive scanning pattern that provides high-density coverage over time, a characteristic particularly beneficial for both SLAM and semantic mapping. Unlike conventional mechanical LiDARs that repeat the same scan lines, the Livox Mid-360’s quasi-random pattern accumulates dense 3D geometry as the sensor moves, ensuring that the entire 360° environment is thoroughly sampled even during short observation periods. This high point density and wide coverage make the Mid-360 well-suited for STCM’s dual requirements of precise metric localization and reliable semantic anchoring via LiDAR-to-camera projection.

Visual perception is provided by a Tier IV C2-176 fisheye camera with 176° field of view, delivering wide-angle RGB imagery for semantic detection and multi-modal fusion. The fisheye lens captures substantially broader coverage than standard perspective cameras, enabling semantic detection of objects in peripheral regions that would otherwise require multiple viewpoints or active camera steering. This wide field of view is especially valuable in corridor and room-corner scenarios where landmarks may appear at large angles from the robot’s forward direction.

The platform is equipped with an AgileX PiPER 6-DOF robotic arm for object manipulation tasks. While manipulation capabilities are not exercised in the current navigation experiments, the arm is integrated into the system architecture to support future assistive tasks such as object retrieval and placement. Communication between the robot’s computing platform and motor controllers is managed through two USB-CAN adapters, providing reliable low-latency interfaces to the drive system and manipulator control boards.

The software stack is organized around ROS 2 Humble as the core middleware for sensor integration, navigation, and system coordination. FAST-LIO2 provides LiDAR-inertial odometry and mapping, delivering real-time pose estimation and metric map construction as described in Section 3.2. Navigation2 (Nav2) serves as the ROS 2 nav-

igation stack for global and local planning, costmap management, and recovery behaviors, enabling the robot to execute waypoint sequences generated from STCM graph plans. MoveIt 2 is installed for manipulation motion planning and will be utilized in future work when assistive manipulation tasks are integrated with navigation. Finally, Octomap_server2 provides 3D occupancy mapping for visualization and collision checking, complementing the point cloud maps produced by FAST-LIO2 with volumetric representations suitable for display in RViz and offline analysis.

4.2. Experimental Environment

Experiments are conducted in the 7th floor of Building 2 at Tokyo Metropolitan University, Hino Campus. The corridor environment presents typical indoor navigation challenges: narrow passages, dynamic lighting from windows, furniture and objects (tables, chairs, trash bins, water fountain, door lifts, bottles, cups, glass windows, cardboard boxes), and occasional human presence.

The corridor is approximately 40 meters in length and 2.5 meters in width, with multiple doorways, open areas with tables and chairs, and a water fountain alcove. This environment is representative of domestic and institutional settings where assistive robots are expected to operate. The presence of transparent surfaces (glass windows) and reflective materials poses challenges for both LiDAR-based SLAM and vision-based semantic grounding.

For STCM evaluation, we define a set of semantic prompts corresponding to objects and landmarks commonly present in the corridor: “table,” “chair,” “trash bins,” “water fountain,” “door,” “bottle,” “cup,” “glass window,” “cardboard box,” “desk,” and “emergency exit sign.” These prompts are provided to GroundingDINO during semantic grounding. The robot performs multiple traversals of the corridor under teleoperation and autonomous navigation, accumulating observations to build the STCM graph online.

4.3. Parameter Configuration

Table 1 summarizes the parameter values used in our implementation, selected based on the spatial scale of the environment and empirical tuning. The GNG threshold $D_{\text{new}} = 0.5$ m produces a fine-grained place graph with dense node coverage, resulting in approximately 60–80 place nodes for the 40-meter corridor. The semantic fusion rate $\alpha = 0.1$ provides stable label accumulation while allowing adaptation to changing observations over roughly 10 visits to a place. Navigation uses Nav2 with Smac Planner 2D for global planning (0.05 m costmap resolution), MPPI controller for local trajectory optimization, maximum linear velocity 0.26 m/s, and maximum angular velocity 1.0 rad/s.

Table 1. STCM parameter configuration

Module	Parameter	Value
Place GNG	D_{new} (node creation)	0.5 m
	ϵ_b (winner learn rate)	0.1
	ϵ_n (neighbor learn rate)	0.01
	a_{max} (edge age threshold)	50
Instance GNG	$D_{\text{new}}^{\text{inst}}$ (instance creation)	0.5 m
	ϵ_w^{inst} (winner learn rate)	0.05
	ϵ_n^{inst} (neighbor learn rate)	0.0006
	$a_{\text{max}}^{\text{inst}}$ (edge age)	200
	D_{gate} (outlier gate)	0.0 m
	n_{min} (min. observations)	3
Semantics	τ_{box} (box confidence)	0.55
	τ_{text} (text matching)	0.55
	α (temporal fusion rate)	0.1



Fig. 7. Top-down 2D projection of the metric LiDAR map $\mathcal{M}$ produced online by FAST-LIO2 during corridor mapping. The visualization provides the metric context in which STCM operates: the FAST-LIO2 pose stream defines the robot trajectory used by place-GNG to create a sparse place graph, while the metric map supports waypoint execution and collision-aware local planning.

5. Experimental Results

5.1. Online Place Graph Construction

We demonstrate online STCM construction during a teleoperated traversal of the corridor environment. The robot begins with an empty place graph and incrementally adds nodes as it explores. Fig. 7 shows a top-down 2D projection of the metric map $\mathcal{M}$ generated online by FAST-LIO2 during a single traversal (approximately 5 minutes of operation). This metric layer provides the map-frame pose stream used by place-GNG to discretize the trajectory into revisitable places. The resulting STCM contains 54 place nodes, 65 place-to-place edges (visualized in Fig. 9), and 19 anchored semantic object instances, capturing the corridor’s main structure: a dense sequence of places along the hallway with branches at doorways and the water fountain alcove.

Node spacing is approximately 0.5–0.8 m, consistent with the $D_{\text{new}} = 0.5$ m threshold and local GNG adaptation (Fig. 9). Edges reflect observed transitions: the main corridor produces a chain structure, while areas where the robot turned around or looped (e.g., approaching the water fountain and returning) produce additional cross-connections.

Table 2. Example semantic scores for STCM place nodes from corridor navigation experiment. Scores are accumulated via temporal fusion ($\alpha = 0.1$) from GroundingDINO detections. The dominant class (highest score) becomes the place label. Visit counts indicate the number of times each place was traversed during the run.

Node	Visits	Semantic Scores	Label
0	49	water fountain: 0.20	water fountain
14	27	trash bins: 0.59, water fountain: 0.49, chair: 0.28	trash bins
22	30	bottle: 0.26, trash bins: 0.10, cup: 0.08	bottle
35	18	chair: 0.14, desk: 0.10	chair
11	11	trash bins: 0.56, cardboard box: 0.38, water fountain: 0.35	trash bins

The graph topology aligns well with human intuition about navigable paths in the environment.

5.2. Semantic Label Fusion and Stability

Semantic labels are accumulated into place nodes over repeated visits. Initially, nodes have no dominant label (uniform zero scores). As the robot observes objects near a place, the corresponding semantic scores increase. Table 2 shows example nodes with their accumulated semantic scores from the experimental corridor navigation run. Representative semantic detection results for different place categories are shown in Fig. 8.

Node labels stabilize over time despite frame-to-frame detection noise. For example, place node 0 near the water fountain area, visited 49 times during the run, accumulates a consistent score for “water fountain” (0.20), becoming the most frequently visited landmark. Place node 14, visited 27 times, shows high scores for “trash bins” (0.59) along with co-occurring objects such as “water fountain” (0.49) and “chair” (0.28), reflecting the spatial proximity of these semantic features in the corridor. Place node 11 demonstrates multi-object accumulation, with scores for “trash bins” (0.56), “cardboard box” (0.38), and “water fountain” (0.35), indicating a location where multiple semantic objects are persistently detected across 11 visits.

The temporal fusion mechanism ($\alpha = 0.1$) effectively filters detection errors. False positives from GroundingDINO (e.g., misclassifying a window reflection as a “bottle”) contribute low confidence and are outweighed by consistent detections of actual objects over multiple observations. This demonstrates the value of accumulating semantic evidence over time rather than relying on single-frame detections.

Fig. 9 visualizes the resulting STCM by plotting the exported graph data (serialized as JSON) in the global map frame. In our run, the exported graph contains 54 place nodes, 65 place edges, and 19 persistent semantic object instances produced by the instance-level GNG. The object instances comprise 11 chairs, 2 cardboard boxes, and single instances of trash bins, bottle, cup, desk, water fountain, and emergency exit sign, reflecting the typical fur-

Table 3. Quantitative comparison of STCM variants. Metrics marked “—” are not defined or not reported for a given method. $\uparrow$ higher is better, $\downarrow$ lower is better.

Method	$ P $	LabelCov $\uparrow$	$ S $	$ E_S $	Mean stability $\uparrow$	$ \Delta S / S_{GT} \downarrow$
Semantic-only	—	—	197	2469	—	9.368
Semantic + place-GNG (no instance GNG)	59	—	28	56	—	0.474
Dual-GNG STCM (proposed)	54	0.796	19	2	1.000	0.263

nishings and features of the indoor corridor environment. The plot is generated directly from the JSON, which also serves as the structured representation read by the LLM during deployment. Place nodes are shown as green circles, with marker size indicating the relative number of visits to each place. Semantic object instances are shown at their anchored 2D positions, and object–place links illustrate the associations used for temporal semantic fusion.

To quantitatively evaluate the contribution of each component in the STCM architecture, we compare the full dual-GNG STCM system against ablated variants. Table 3 reports key metrics across three configurations: a semantic-only approach that performs open-vocabulary detection without topological abstraction, a semantic mapping system augmented with place-GNG but without instance-level GNG fusion, and the proposed dual-GNG STCM with both place-level and instance-level GNG operating online. The metrics capture graph structure ($|P|$ = number of place nodes) and object-level structure ($|S|$ = number of semantic object instances). LabelCov measures the fraction of place nodes with stable semantic labels, where a label is considered stable if its confidence score $\max_c s_{i,c}$ exceeds 0.55 after sufficient observations. Mean stability quantifies instance position consistency by computing the standard deviation of instance centroid positions across revisits; instances with deviations below 0.15 m are deemed stable. The erroneous instance count $|E_S|$ tallies semantic instances that persist in the map but do not correspond to actual physical objects in the environment. Finally, semantic cardinality deviation $|\Delta S|/|S_{GT}|$ measures the relative difference between the number of detected instances and the ground-truth count $|S_{GT}|$ for the corridor.

Table 3 suggests that the dual-GNG architecture is important for controlling duplicate instance creation: the semantic-only approach yields 197 semantic object instances with 2469 erroneous duplicates, producing a large semantic cardinality deviation ($|\Delta S|/|S_{GT}| = 9.368$), which is consistent with the lack of spatial clustering and the resulting redundancy under viewpoint variation and detection noise. Adding place-GNG without instance-GNG reduces over-segmentation (28 instances, 56 erroneous instances; $|\Delta S|/|S_{GT}| = 0.474$) and yields a place graph ($|P| = 59$), while place-level semantic coverage (LabelCov) is not reported for this ablation. In contrast, the proposed dual-GNG STCM reports $|P| = 54$ and achieves LabelCov of 0.796 with a mean stability of 1.000 (all instances below the 0.15 m position standard-deviation threshold across revisits), while reducing the in-

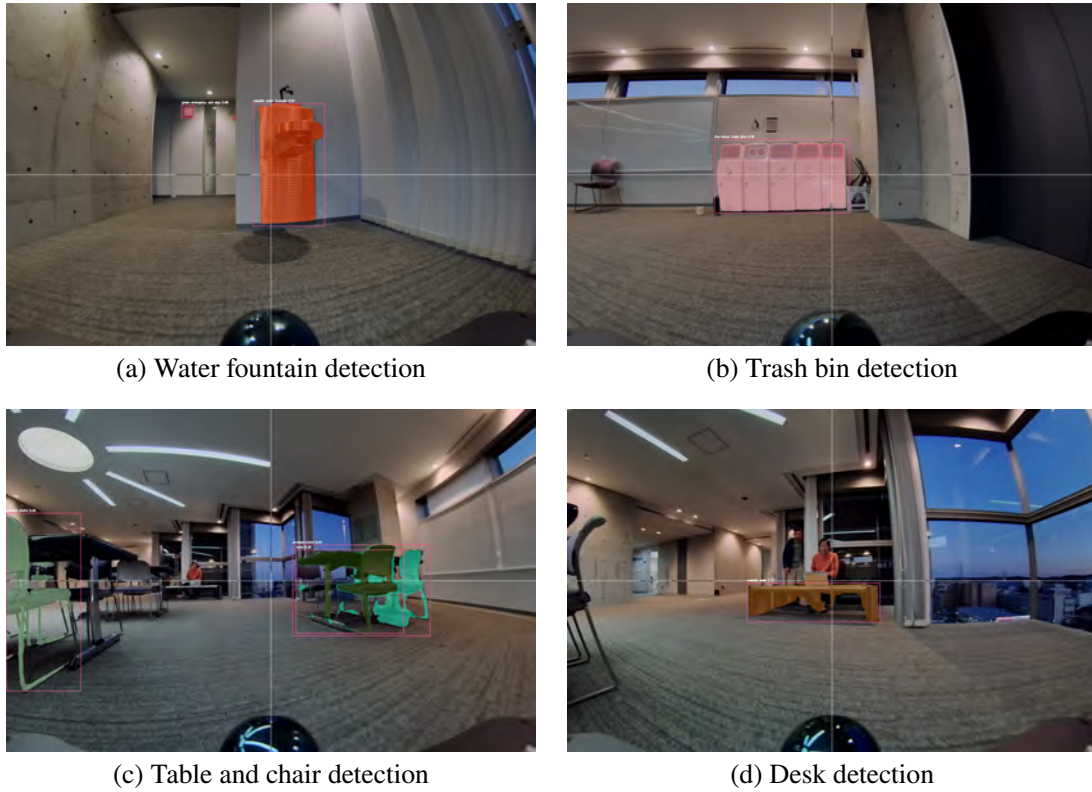


Fig. 8. Representative semantic detection results using GroundingDINO for open-vocabulary object detection in the corridor environment. Detections are temporally fused into STCM place nodes to create stable semantic labels: (a) water fountain area, (b) trash bin area, (c) table/chair area, and (d) desk area.

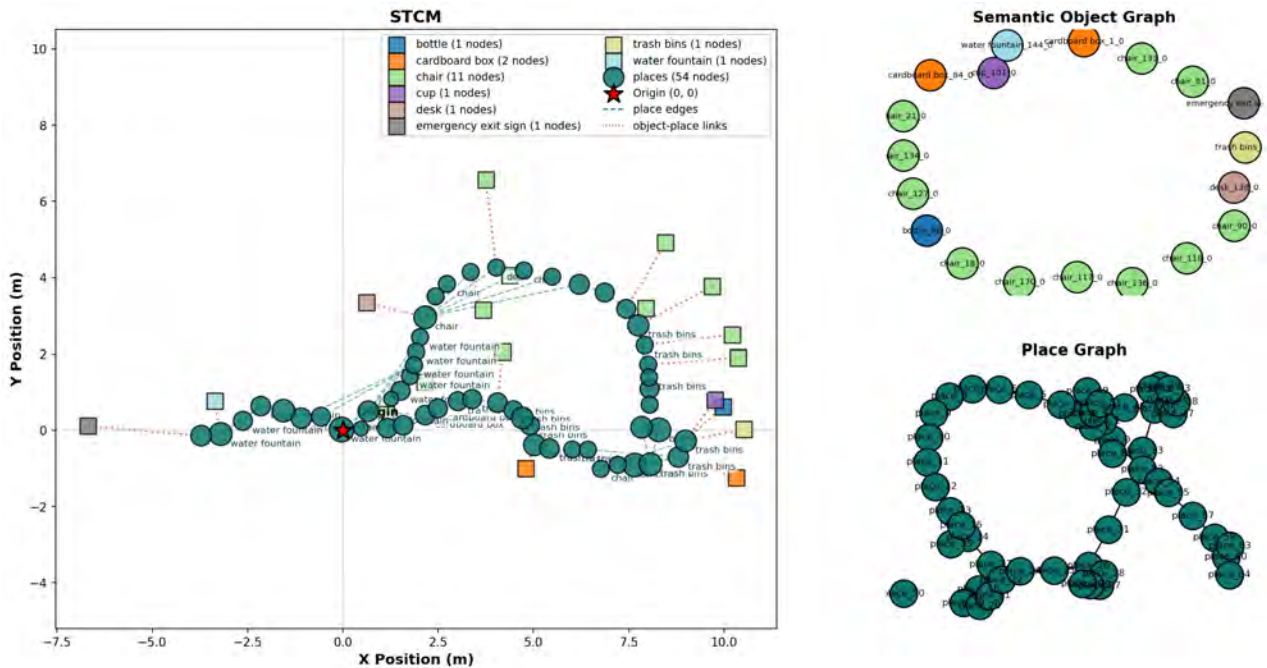


Fig. 9. STCM visualization reconstructed from the exported JSON graph. Place nodes are plotted as green circles in the FAST-LIO2 map frame, with larger circles indicating places that were revisited more frequently. Semantically grounded object instances (GroundingDINO + SAM) are overlaid at their anchored 2D positions, and edges depict both place-to-place connectivity and object-to-place associations, enabling inspection of topological structure, semantic consistency, and graph serializability for downstream consumption (e.g., by an LLM).

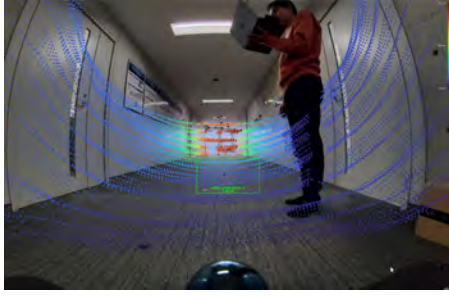


Fig. 10. Multi-modal fisheye-LiDAR sensor fusion. The 176° FOV fisheye camera captures wide-angle RGB imagery while the Livox Mid-360 provides dense 3D LiDAR point clouds. The system projects LiDAR points into the image frame (per-point (u, v)) so that segmented detections can be associated directly with 3D LiDAR returns and then transformed to the map frame via TF, supporting robust place labeling.

stance count to 19 with $|E_S| = 2$ erroneous instances. Although the remaining cardinality deviation (0.263) indicates a residual mismatch in instance cardinality relative to the ground truth, these results indicate that combining place-level and instance-level GNG supports stable online semantic topological mapping in the evaluated setting.

5.3. Multi-Modal Sensor Fusion: Fisheye-LiDAR Integration

The wide-angle fisheye camera (176° FOV) provides substantially broader coverage than standard perspective cameras, enabling semantic detection of objects in peripheral regions that would otherwise require multiple viewpoints. LiDAR-camera fusion produces colorized point clouds that support both geometric mapping and semantic anchoring.

Fig. 10 illustrates synchronized fisheye imagery and LiDAR point clouds. The fisheye lens captures the full corridor width and overhead structure in a single frame, while the Livox Mid-360's non-repetitive scan accumulates dense 3D geometry. The system projects LiDAR points into the image frame and associates segmented detections (from GroundingDINO + SAM) with the subset of 3D points whose (u, v) lie inside the mask, producing 3D-anchored semantic labels that are then transformed to the map frame via TF.

This multi-modal fusion improves robustness: the fisheye camera provides rich appearance cues for semantic classification, while LiDAR provides accurate metric positions unaffected by lighting variation or lens distortion. The combination enables reliable place labeling even in challenging conditions (e.g., backlit windows, shadowed corners).

5.4. Autonomous Navigation with STCM

We evaluate autonomous navigation using the constructed STCM graph. The robot is commanded to navigate to a place specified by semantic label (e.g., “go to the

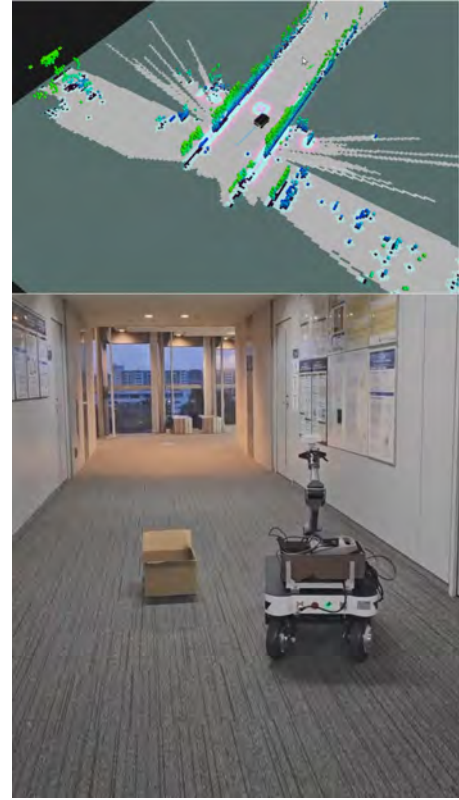


Fig. 11. Autonomous navigation with STCM. The robot navigates to a semantically specified goal (“go to the table area”) by planning over the STCM place graph and executing via ROS 2 Nav2 waypoint following. The topological graph provides stable high-level waypoints while Nav2 handles metric planning and local obstacle avoidance.

table area”). The system identifies the place node with the highest score for “table,” computes a graph path from the current node to the target node, and executes the path via Nav2 waypoint following.

Fig. 11 shows a representative navigation trial. The robot successfully navigates from its starting position to the target place, following the topological path through intermediate nodes and using metric planning to avoid obstacles (e.g., a human crossing the corridor, a temporarily placed box). Nav2's dynamic replanning handles local perturbations, while the topological graph provides stable waypoints that guide the global trajectory.

Across 10 navigation trials to different semantic goals (table, water fountain, doorway), the robot achieved 9 successes and 1 failure. The failure case occurred when the target place label (“cardboard box”) was ambiguous (two nodes had similar scores for this label) and the robot navigated to the incorrect node. This highlights the importance of semantic label disambiguation and potentially finer-grained place representation for assistive tasks.

5.5. Human Obstacle Detection and Avoidance

In assistive navigation scenarios, safe interaction with humans is critical. The Livox Mid-360 LiDAR reliably detects human obstacles in the corridor, and Nav2's MPPI



Fig. 12. Human obstacle detection and avoidance during STCM-guided navigation. The Livox Mid-360 LiDAR detects the human in the corridor, and Nav2’s MPPI controller computes a collision-free trajectory that maintains safe clearance. The STCM topological plan remains stable while metric planning handles the dynamic obstacle.

controller computes collision-free trajectories by optimizing sampled rollouts over the costmap. Fig. 12 shows the robot navigating around a human crossing its path. The costmap (constructed from LiDAR scans via the SpatioTemporalVoxelLayer) marks the human as an obstacle, and the MPPI controller generates a trajectory that slows and deviates laterally to maintain clearance while minimizing path deviation and control effort.

The STCM graph remains stable during such local perturbations: the robot’s high-level topological plan is unchanged, while metric planning handles dynamic obstacles. This demonstrates the benefit of the dual representation, where symbolic place goals are robust to transient environmental changes, while reactive control ensures safety.

5.6. Place Node Stability Analysis

To quantitatively assess the spatial distribution and coverage of semantic labels across the corridor environment, we analyze the position statistics of place nodes grouped by semantic label. Table 4 reports the number of place nodes with each label, total visit counts accumulated across all nodes of that label, and position standard deviations σ_x and σ_y that characterize the spatial extent of each semantic category.

The spatial distribution statistics reveal several key in-

Table 4. Spatial distribution statistics for semantic labels in the STCM place graph. Standard deviations reflect the spatial extent over which each label appears in the corridor. Labels appearing at multiple distinct locations (e.g., trash bins at 18 nodes) show larger σ values, while labels appearing at a single location show $\sigma \approx 0$.

Semantic Label	σ_x (m)	σ_y (m)	Nodes	Total Visits	Avg. Visits/Node
Trash bins	1.85	1.04	18	118	6.6
Water fountain	1.84	0.80	12	99	8.2
Bottle	0.00	0.00	1	30	30.0
Chair	1.81	2.07	4	21	5.2
Cardboard box	0.17	0.09	2	7	3.5
Desk	0.00	0.00	1	1	1.0

Note: σ is computed from the positions of all place nodes labeled with each semantic category. Single-node labels have $\sigma = 0$ by definition.

sights about semantic label coverage and stability. Frequently occurring environmental features such as trash bins (18 nodes, 118 total visits) and water fountains (12 nodes, 99 total visits) are detected at multiple distinct locations throughout the corridor, with spatial extents $\sigma_x \approx 1.8$ m and $\sigma_y \approx 0.8$ –1.0 m reflecting the distribution of these objects along the corridor topology. These values indicate that the STCM successfully captures the spatial spread of common corridor fixtures, enabling the robot to ground commands like “find a trash bin” to multiple candidate locations rather than a single fixed position.

The average visits per node metric quantifies the benefit of repeated traversals and temporal evidence accumulation. The single bottle node accumulated 30 visits, the highest per-node average, indicating that the robot repeatedly passed through this location and consistently reinforced the semantic label through temporal fusion. In contrast, water fountain and trash bin nodes show moderate per-node averages (8.2 and 6.6 respectively), as visits are distributed across multiple spatial instances of these categories. This difference reflects the underlying environment structure: unique objects accumulate all revisit evidence at one node, while distributed object categories spread evidence across their spatial instances.

A critical distinction emerges between localized and distributed labels. Labels appearing at a single location (bottle, desk) exhibit $\sigma \approx 0$ by definition, while labels appearing at multiple locations (chair, cardboard box, trash bins, water fountain) show non-zero standard deviations that characterize their spatial distribution. This distinction has important implications for navigation planning: localized labels serve as unique landmarks that provide unambiguous goal targets, while distributed labels require disambiguation through additional context (e.g., “go to the nearest trash bin” vs. “go to the trash bin near the water fountain”). Future work may incorporate spatial-relational reasoning to automatically resolve such ambiguities using the place graph topology.

6. Discussion

6.1. Design Rationale and Trade-Offs

The STCM architecture reflects a deliberate trade-off between representational richness and practical integration. Dense semantic mapping approaches (e.g., ConceptFusion, VLMaps) provide high-resolution spatial queries and fine-grained object-level reasoning, but often require substantial computational resources and may not directly interface with standard navigation stacks. Full scene graph systems (e.g., Hydra, S-Graphs+) support rich relational reasoning but emphasize global optimization and may not prioritize temporal fusion of open-vocabulary detections or lightweight goal grounding. Prior evaluations of topological mapping methods have shown that the best approaches can build graphs with less than 5% localization error while running in real-time [41], demonstrating the practical viability of topological abstractions for navigation.

STCM prioritizes interpretability, online learnability, and ROS 2 compatibility. The place graph is sparse (tens of nodes rather than thousands of landmarks), enabling efficient graph search and transparent explanation of navigation decisions to users. Semantic labels are accumulated over time via simple exponential smoothing, avoiding the need for complex probabilistic inference or offline training. The graph is directly serializable and visualizable, supporting debugging and user inspection. This aligns with the cognitive map concept from the Spatial Semantic Hierarchy theory [42], which posits that a topological graph with place names forms a human-like cognitive map and predicts that having a semantic-topological map greatly enhances an agent's ability to plan and explain routes [42].

From an ecological psychology perspective, behavior generation depends on how an individual's perceptual systems detect affordances, which are actionable properties of the environment that support specific activities [1, 5]. In STCM, place nodes implicitly capture navigational affordances: a node labeled "doorway" affords passage between rooms, while a node labeled "table" affords approach for object manipulation. The multi-scopic organization separates concerns by temporal and spatial scale: microscopic processes detect immediate affordances (e.g., a suddenly obstructed foothold), mesoscopic processes accumulate affordances into place representations (e.g., recognizing that a location consistently affords visibility of the water fountain), and macroscopic processes reason about affordance sequences for goal achievement (e.g., planning a route through a series of navigable places). This separation enables efficient computational resource allocation: the system dedicates high-frequency, low-latency processing to immediate sensorimotor demands while deferring global reasoning to lower-frequency updates. The resulting architecture mirrors biological cognitive systems, where perception, action, and cognition operate at distinct but coupled timescales [1], and achieves both adaptability to dynamic perturbations and optimality in long-horizon navigation.

The choice of GNG for topological learning provides

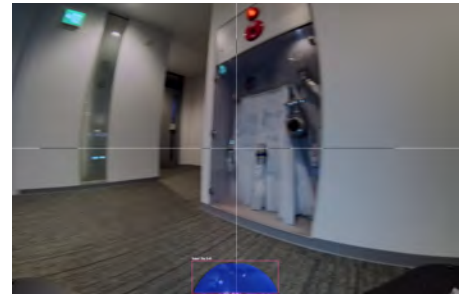


Fig. 13. Failure case in semantic grounding. Motion blur, occlusion, and challenging lighting conditions degrade GroundingDINO detection quality, resulting in missed or incorrect object labels. Temporal fusion mitigates single-frame errors but requires multiple observations to stabilize labels. Future work should incorporate more stable image capture and confidence-aware fusion strategies.

automatic node creation and topology adaptation without requiring manual place annotation or supervised training. GNG's incremental nature aligns well with lifelong learning scenarios where the robot continuously explores and updates its map. Multi-scale variants of GNG [26] can be integrated in future work to support hierarchical place abstractions (e.g., room-level and building-level graphs).

The use of GroundingDINO for open-vocabulary detection enables flexible semantic prompting without retraining. Users can specify task-relevant concepts ("medicine cabinet," "bedside") that may not be present in standard object detection datasets. SAM refinement improves spatial precision but adds computational cost; in our implementation, we apply SAM selectively (e.g., for high-confidence detections or when disambiguation is needed).

6.2. Semantic Grounding Challenges

Despite the promise of foundation models, semantic grounding remains challenging in real indoor environments. GroundingDINO detection quality varies with object appearance, occlusion, and prompt phrasing. Small or partially occluded objects (e.g., a cup on a table behind a chair) may be missed, while visually similar categories (e.g., "trash bin" vs. "cardboard box") can be confused. Transparent and reflective surfaces (glass windows, polished floors) produce detection artifacts. As a large model, GroundingDINO requires significant computational resources and can still fail on unusual phrasing or abstract semantic queries [27]. Similarly, while SAM provides powerful segmentation capabilities, it can over-segment or under-segment in complex scenes and was trained primarily on daily objects, potentially missing specialized indoor landmarks [28].

Fig. 13 illustrates a representative failure mode where motion blur and challenging lighting conditions degrade open-vocabulary detection performance. Such failure cases highlight the importance of temporal fusion: single-frame errors are mitigated by accumulating evidence over multiple observations, but initial detection failures can delay or prevent place label assignment.

Temporal fusion into place nodes mitigates single-frame errors but introduces latency: a place label becomes stable only after multiple observations. In dynamic environments where objects move frequently, this may lead to stale labels. Future work could incorporate explicit object tracking and change detection to distinguish persistent place features (“kitchen,” “doorway”) from transient objects (“cup on table”).

Range-based 3D grounding via image-projected LiDAR provides reliable metric anchors but introduces sparsity and calibration sensitivities. LiDAR beams may miss small or thin objects, and imperfect camera–LiDAR calibration can cause projected points to fall outside a segmentation mask. Robust association and outlier handling remain important; in particular, sparse returns can cause unstable 3D anchors for slender structures (e.g., chair legs). When needed, a dense monocular depth estimator (Depth Anything [29]) can be used as an optional fallback for masks with no LiDAR hits, but it requires careful validation of scale and failure modes. Depth Anything provides relative depth predictions that must be scaled to metric units, typically via a scale factor estimated from LiDAR correspondences or prior calibration, and its accuracy degrades for distant objects or scenes with large depth discontinuities. The depth fallback mechanism was not employed in the experiments reported in this paper.

6.3. STCM for Assistive Human-Robot Interaction

The primary motivation for STCM is to enable assistive navigation in domestic and care environments, where users are often non-expert and expect natural, interpretable interaction. Assistive indoor mobile robots must navigate spaces that are fundamentally structured around human activity: places are identified by function and affordances rather than metric coordinates, and goals are communicated through familiar labels such as “kitchen,” “bedside,” or “near the medicine cabinet.” Traditional metric SLAM provides the geometric foundation for safe motion, but it does not directly support the symbolic, language-mediated communication that characterizes human spatial cognition and instruction giving. STCM bridges this gap by coupling a metric map with an explicit place graph enriched with open-vocabulary semantic labels, creating a representation that is simultaneously grounded in reliable geometric control and aligned with how people naturally describe and reason about indoor spaces.

The STCM architecture supports assistive human-robot interaction through four key mechanisms that address the practical requirements of domestic robot deployment. First, STCM enables language grounding by providing an explicit mapping from place labels to navigation targets. Users can specify goals as familiar place names rather than coordinates or waypoints, and the system resolves these requests by identifying nodes in the place graph whose semantic labels match the user’s description. Early frameworks for interpreting navigation instructions demonstrated that given a semantic map, robots can effec-

tively follow natural language directions [43], and STCM’s integration of open-vocabulary detection extends this capability to arbitrary user-specified concepts that may not be present in standard object detection datasets. This flexibility is critical in personalized assistive scenarios, where task-relevant landmarks such as “medicine cabinet,” “favorite reading chair,” or “bedside table” must be learned and recognized without task-specific retraining.

Second, STCM prioritizes transparency and explainability, qualities that research has shown to be central to user trust and acceptance in human-robot interaction. The place graph and semantic scores are directly inspectable and visualizable, enabling users to see which places the robot has learned, what labels it has assigned, and why a particular navigation decision was made. For example, when the robot navigates to a place labeled “kitchen,” it can explain that the decision was based on high accumulated scores for “sink” and “stove,” making the reasoning process explicit and verifiable. Research has shown that transparency fosters trust: users preferred robots that explained their route choices and reacted more positively to systems with natural language justifications [44]. Unlike black-box policies or end-to-end learned navigation systems that map sensor inputs directly to control actions, STCM’s decision process can be exposed at the level of the graph and verbalized in natural language. Prior user studies correlate such symbolic, interpretable representations with increased trust and acceptance in human-robot interaction [44], and reviews of explainable AI techniques in robotics highlight that symbolic representations and natural language explanations are among the most promising approaches for making robot decision-making transparent to lay users [45]. By maintaining an explicit, inspectable place graph rather than relying on latent embeddings or opaque feature maps, STCM enables users to audit and understand the robot’s spatial knowledge, supporting informed collaboration and corrective feedback.

Third, STCM supports incremental learning without requiring full environment models or offline training. The system is updated online as the robot operates: place nodes are discovered via GNG from the streaming SLAM trajectory, semantic labels are accumulated through temporal fusion of open-vocabulary detections, and edges are refined based on observed traversability. This incremental learning aligns with realistic deployment scenarios where the environment is not fully known in advance and where the robot must adapt to changes over time. As the robot explores, it builds a cognitive map that reflects its actual navigation experience rather than relying on pre-programmed waypoints or hand-annotated semantic labels. This approach mirrors biological spatial learning, where cognitive maps are incrementally constructed through exploration and updated based on experience [3, 4].

6.4. Limitations and Future Directions

Several limitations remain to be addressed in future work. The STCM graph inherits any drift or errors from the underlying SLAM system, and loop closure events in

SLAM can cause global map adjustments that potentially invalidate place node positions and semantic anchors. The current system does not explicitly distinguish static scene structure from movable objects. A place labeled “table area” may accumulate scores for transient objects such as “cup” or “book,” leading to label drift if those objects are removed. Explicit object tracking and temporal decay mechanisms could address this by distinguishing persistent environmental features from temporary object placements, allowing the system to maintain stable place labels while adapting to legitimate environmental changes.

Multi-scopic approaches [39] suggest the value of hierarchical graphs, such as corridor-level, floor-level, and building-level representations, for scalable planning and explanation in larger environments. Extending GNG to multi-scale batch learning [26] is a natural next step that would enable the system to represent and reason about spatial structure at multiple levels of granularity, supporting more efficient long-range navigation and more intuitive explanation of navigation strategies to users.

7. Conclusion

We have presented a semantic topological cognitive map for assistive indoor navigation that addresses the gap between metric SLAM and language-driven human-robot interaction. The STCM system integrates FAST-LIO2 metric localization with a semantic topological place graph learned online via Growing Neural Gas, enriched with open-vocabulary semantic labels from GroundingDINO and Segment Anything Model, and executed through ROS 2 Nav2 for reliable waypoint navigation. Organized conceptually along a multi-scopic neuro-cognitive framework inspired by biological cognitive systems, STCM coordinates intelligent functions across microscopic real-time sensing and reactive control, mesoscopic topological learning and localization, and macroscopic cognitive mapping and deliberative planning. The dual GNG-based learning architecture employs place-level GNG to convert SLAM trajectories into revisitable place nodes and traversability edges, and instance-level GNG to adaptively cluster multiple instances of the same object class with online outlier filtering and temporal evidence accumulation, both operating without offline annotation or supervised training. The open-vocabulary semantic grounding pipeline anchors detections in the map frame and temporally fuses them into persistent node-level semantic scores, enabling the system to recognize and label affordances beyond predefined object categories. Experimental results on a Ranger Garden Assistant platform in corridor environments demonstrate online graph construction with approximately 0.5–0.8 m node spacing, stable semantic label accumulation over repeated visits with position consistency below 0.15 m, multi-modal fisheye-LiDAR fusion enabling wide-angle semantic detection, and successful autonomous navigation to language-specified goals with 9 out of 10 trials reaching the correct semantically identified place.

The STCM architecture represents a practical middle ground between dense semantic maps and full scene graphs, preserving reliable metric control for collision avoidance while maintaining an explicit, online-updated place graph with interpretable semantic labels suitable for assistive dialogue and transparent explanation of navigation decisions to non-expert users. The system aligns with cognitive map theories from psychology and neuroscience, ecological perspectives on affordance-based perception and action, and multi-scopic neuro-cognitive frameworks that organize intelligent functions across coupled temporal and spatial scales, providing a principled foundation for assistive navigation that mirrors biological cognitive systems in its integration of perception, action, and cognition at distinct but interconnected levels. Future work should address SLAM drift compensation and loop closure integration to maintain topological consistency during global map optimization, develop explicit object tracking and change detection to distinguish persistent scene structure from transient objects, and extend the system to hierarchical multi-scale graph representations for scalable planning in larger environments. User studies with older adults and individuals with mobility impairments in realistic home environments are essential to validate assistive task effectiveness, measure trust and acceptance, and evaluate the impact of interpretable place graphs on human-robot interaction quality.

Acknowledgments

This work was partially supported by Japan Science and Technology Agency (JST), Moonshot R&D, with grant number JP-MJMS2034 and TMU local 5G research support.

References

- [1] A. A. Saputra, K. Wada, S. Masuda, and N. Kubota, “Multi-scopic neuro-cognitive adaptation for legged locomotion robots,” *Scientific Reports*, Vol.12, p. 16222, 2022. <https://doi.org/10.1038/s41598-022-19599-2>
- [2] P. A. Hancock, D. R. Billings, K. E. Schaefer, J. Y. C. Chen, E. J. de Visser, and R. Parasuraman, “A meta-analysis of factors affecting trust in human-robot interaction,” *Human Factors*, Vol.53, No.5, pp. 517-527, 2011. <https://doi.org/10.1177/0018720811417254>
- [3] E. C. Tolman, “Cognitive maps in rats and men,” *Psychological Review*, Vol.55, No.4, pp. 189-208, 1948. <https://doi.org/10.1037/h0061626>
- [4] J. O’Keefe and L. Nadel, *The Hippocampus as a Cognitive Map*. Oxford, U.K.: Clarendon Press, 1978.
- [5] J. J. Gibson, *The Ecological Approach to Visual Perception*. Boston, MA, USA: Houghton Mifflin, 1979.
- [6] J. Woo and N. Kubota, “Recognition of indoor environment by robot partner using conversation,” *J. Adv. Comput. Intell. Intell. Inform.*, Vol.17, No.5, pp. 753-760, 2013. <https://doi.org/10.20965/jaciii.2013.p0753>
- [7] N. Kubota and A. Yorita, “Topological environment reconstruction in informationally structured space for pocket robot partners,” *Proc. IEEE Int. Symp. on Computational Intelligence in Robotics and Automation (CIRA)*, pp. 165-170, 2009.
- [8] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, “Kimera: an open-source library for real-time metric-semantic localization and mapping,” *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2020. <https://doi.org/10.1109/ICRA40945.2020.9196885>
- [9] N. Hughes et al., “Hydra: A real-time spatial perception system for 3D scene graph construction and optimization,” *Proc. Robotics: Science and Systems (RSS)*, 2022. <https://doi.org/10.48550/arXiv.2201.13360>

- [10] C. Bavle et al., "S-Graphs+: Real-time semantic SLAM with hierarchical map representation," 2023. <https://doi.org/10.48550/arXiv.2212.11770>
- [11] H. Huang et al., "VLMs: Visual language maps for robot navigation," Proc. IEEE Int. Conf. on Robotics and Automation (ICRA), 2023, originally on arXiv 2022. <https://doi.org/10.48550/arXiv.2210.05714>
- [12] S. Jatavallabhula et al., "ConceptFusion: Open-set multimodal 3D mapping," Proc. Robotics: Science and Systems (RSS), 2023. <https://doi.org/10.48550/arXiv.2302.07241>
- [13] J. Gu et al., "ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning," 2023. <https://doi.org/10.48550/arXiv.2309.16650>
- [14] S. Busch et al., "One map to find them all: Real-time open-vocabulary mapping for zero-shot multi-object navigation," 2024. <https://doi.org/10.48550/arXiv.2409.11764>
- [15] B. Chen et al., "Open-vocabulary queryable scene representations for real world planning," 2022. <https://doi.org/10.48550/arXiv.2209.09874>
- [16] D. Shah et al., "LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action," Proc. Conf. on Robot Learning (CoRL), 2022. <https://doi.org/10.48550/arXiv.2207.04429>
- [17] N. Chang et al., "GOAT: GO to any thing," 2023. <https://doi.org/10.48550/arXiv.2311.06430>
- [18] P. Tang et al., "OpenIN: Open-vocabulary instance-oriented navigation in dynamic domestic environments," 2025. <https://doi.org/10.48550/arXiv.2501.04279>
- [19] S. Raychaudhuri et al., "Zero-shot object-centric instruction following: Integrating foundation models with traditional navigation," 2025. <https://doi.org/10.48550/arXiv.2411.07848>
- [20] L. Zhang et al., "MapNav: A novel memory representation via annotated semantic maps for VLM-based vision-and-language navigation," 2025. <https://doi.org/10.48550/arXiv.2502.13451>
- [21] D. S. Chaplot et al., "Neural topological SLAM for visual navigation," Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2020. <https://doi.org/10.1109/CVPR42600.2020.01289>
- [22] D. An et al., "ETPNav: Evolving topological planning for vision-language navigation in continuous environments," 2023. <https://doi.org/10.48550/arXiv.2304.03047>
- [23] R. Garg et al., "RoboHop: Segment-based topological map representation for open-world visual navigation," 2024. <https://doi.org/10.48550/arXiv.2405.05792>
- [24] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," IEEE Trans. Robotics, Vol.38, No.4, pp. 2053-2073, 2022, originally on arXiv 2021. <https://doi.org/10.1109/TRO.2022.3141876>
- [25] B. Fritzke, "A growing neural gas network learns topologies," Advances in Neural Information Processing Systems (NeurIPS), Vol.7, pp. 625-632, 1995.
- [26] F. Ardilla, S. Saputra, and N. Kubota, "Multi-scale batch-learning growing neural gas efficiently for dynamic data distributions," Int. J. Autom. Technol., Vol.17, No.3, pp. 206-216, 2023. <https://doi.org/10.20965/ijat.2023.p0206>
- [27] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," 2023. <https://doi.org/10.48550/arXiv.2303.05499>
- [28] A. Kirillov et al., "Segment anything," 2023. <https://doi.org/10.48550/arXiv.2304.02643>
- [29] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), pp. 10 371-10 381, 2024.
- [30] S. Macenski et al., "The Marathon 2: A navigation system," Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS), 2020. <https://doi.org/10.1109/IROS45743.2020.9341207>
- [31] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruan, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as I can, not as I say: Grounding language in robotic affordances," Proc. Conf. on Robot Learning (CoRL), 2022. <https://doi.org/10.48550/arXiv.2204.01691>
- [32] H. Zhang, H. Deng, J. Ou, and C. Feng, "Mitigating spatial hallucination in large language models for path planning via prompt engineering," Sci. Rep., Vol.15, p. 8881, 2025. <https://doi.org/10.1038/s41598-025-93601-5>
- [33] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3D scene graph: A structure for unified semantics, 3D space, and camera," Proc. IEEE Int. Conf. on Computer Vision (ICCV), pp. 5664-5673, 2019. <https://doi.org/10.1109/ICCV.2019.00576>
- [34] S. Kim et al., "3D scene graph for intelligent agent memory," IEEE Trans. Cybern., Vol.51, No.6, pp. 3146-3159, 2021, published online 2019. <https://doi.org/10.1109/TCYB.2019.2935974>
- [35] S. H. Allu, I. Kadosh, T. Summers, and Y. Xiang, "A modular robotic system for autonomous exploration and semantic updating in large-scale indoor environments," 2025. <https://doi.org/10.48550/arXiv.2409.15493>
- [36] K. Yokoyama et al., "VLFM: Vision-language frontier maps for zero-shot semantic navigation," 2023. <https://doi.org/10.48550/arXiv.2312.03275>
- [37] D. S. Chaplot, D. Gandhi, S. Gupta, A. Gupta, and R. Salakhutdinov, "Learning to explore using active neural SLAM," Proc. Int. Conf. on Learning Representations (ICLR), 2020, also presented at NeurIPS 2020 as Object Goal Navigation using Goal-Oriented Semantic Exploration. <https://openreview.net/forum?id=HklXn1BKDH>
- [38] N. Zantout, H. Zhang, P. Kachana, J. Qiu, G. Chen, J. Zhang, and W. Wang, "SORT3D: Spatial object-centric reasoning toolbox for zero-shot 3D grounding using large language models," 2025. <https://doi.org/10.48550/arXiv.2504.18684>
- [39] N. Kubota, C. Z. Siow, and T. Obo, "Multiscopic topological twin for cyber-physical systems," in AI-Driven Smart Industrial Technologies. IGI Global, 2026, pp. 151-174. <https://doi.org/10.4018/979-8-3693-7994-3.ch007>
- [40] A. Junaedy, H. Masuta, Y. Fuse, K. Sawai, T. Motoyoshi, and N. Takagi, "Online topological mapping on a quadcopter with fast growing neural gas," J. Adv. Comput. Intell. Inform., Vol.28, No.6, pp. 1354-1366, 2024. <https://doi.org/10.20965/jaciii.2024.p1354>
- [41] K. Muravyev and K. Yakovlev, "Evaluation of topological mapping methods in indoor environments," IEEE Access, Vol.11, pp. 132 683-132 698, 2023. <https://doi.org/10.1109/ACCESS.2023.3335818>
- [42] B. Kuipers, "The spatial semantic hierarchy," Artif. Intell., Vol. 119, No. 1-2, pp. 191-233, 2000. [https://doi.org/10.1016/S0004-3702\(00\)00017-5](https://doi.org/10.1016/S0004-3702(00)00017-5)
- [43] S. Tellex, T. Kollar, S. Dickerson, M. R. Walter, A. G. Banerjee, S. Teller, and N. Roy, "Understanding natural language commands for robotic navigation and mobile manipulation," Proc. AAAI Conf. on Artificial Intelligence, pp. 1507-1514, 2011.
- [44] O. Sotomi, D. Kodi, and A. Arab, "Trust through transparency: Explainable social navigation for autonomous mobile robots via vision-language models," 2025. <https://doi.org/10.48550/arXiv.2504.05477>
- [45] M. Stange, P. Altheld, F. Hoppe, and O. von Stryk, "Explainable AI for robotics: A survey," Front. Robot. AI, Vol.8, p. 612743, 2021. <https://doi.org/10.3389/frobt.2021.612743>